

Visualization with Matplotlib & Pandas

Overview

- [Visualization](#)
- [Matplotlib](#)
- [Pandas](#)
- [Plot Polishing](#)
- [Takeaways](#)

Visualization

- *Visualization* refers to creating figures or *plots* to create visual representations of data or analytic results.
- Visualization is useful at all stages of data analysis:
 - communication,
 - exploration,
 - modeling.

Matplotlib

- [Matplotlib](#) is the core visualization library in Python.
- Other useful plotting libraries use Matplotlib as a back-end:
 - pandas
 - seaborn
 - plotnine.

Matplotlib

- Other useful plotting libraries use Matplotlib as a back-end.
- Familiarity with the Matplotlib API will help you to understand these libraries and will also be useful for customization.

Matplotlib

- In these slides, we'll focus on the object-oriented interface to matplotlib and the *pyplot* API.
- These are primarily *imperative* APIs -- you say *what to do*.
- Contrast with plotnine which is a *declarative* API -- you say *what you want*.

Canonical Import

- Import the pyplot API as `plt` and (when needed) matplotlib as `mpl`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
mpl.__version__
```

Example Data

- Below we create 3 NumPy ndarray objects with **y** and **z** having a specified correlation with **x**.

```
rng = np.random.default_rng(9 * 26 * 2021)
n = 100
rho = .3, .7
x = rng.normal(size=n)
y = 2 + rho[0] * x + np.sqrt(1 - rho[0] ** 2) * rng.normal(size=n)
z = np.pi + rho[1] * x + np.sqrt(1 - rho[1] ** 2) * rng.normal(size=n)
df = pd.DataFrame({'x': x, 'y': y, 'z': z})
df
```

Figures and Axes

- Plotting is done within a **Figure** object, `plt.figure()`.
- *Subplots* or *axes* are added with `.add_subplot()`.
- The outputs are *handles* for referencing the objects created.

```
fig0 = plt.figure()
ax0 = fig0.add_subplot(3, 1, 1)
ax1 = fig0.add_subplot(3, 1, 2)
ax2 = fig0.add_subplot(3, 1, 3)
fig0.tight_layout()
_ = plt.scatter(x, y, color='darkgreen')
_ = plt.scatter(x, z, color='red')
_ = ax0.hist(x)
_ = ax1.hist(y, color='darkgreen', alpha=0.5)
_ = ax1.hist(z, color='red', alpha=0.5)
```

Layout and Spacing

- Control spacing around subplots using `fig.subplots_adjust()`
- I find that `fig.tight_layout()` tends to produce nice spacing.

Figures and Axes

- `plt.subplots()` simplifies creating a figure with a given subplot layout.
- Use `sharex` and `sharey` to reduce clutter when plotting on a common scale.

```
fig1, axes = plt.subplots(nrows=3, ncols=1, sharex=True)
fig1.tight_layout()
_ = axes[0].hist(x)
_ = axes[1].scatter(x, y, color='darkgreen')
_ = axes[2].scatter(x, z, color='red')
_, _ = axes[1].set_ylim(-2, 6), axes[2].set_ylim(-2, 6)
_ = axes[2].set_xlabel('x')
for i, lab in enumerate(['count', 'y', 'z']):
    _ = axes[i].set_ylabel(lab)
```

Scatter

- Scatterplots (`plt.scatter()`) are useful for showing the relationship between two variables.
- Label x or y axis using `.set_*label()` method of an axes.
- Use the `label` parameter in a plotting *artist* for a simple legend.

```
fig2, ax2 = plt.subplots(nrows=1, ncols=1)
plt.scatter(x, y, color='darkgreen', alpha=0.5, label='y')
plt.scatter(x, z, color='red', alpha=0.5, label='z')
_, _ = ax2.set_xlabel('x (IV)'), ax2.set_ylabel('y or z (DVs)')
_ = ax2.legend(loc='upper left')
```

Error Bars

- It's often useful to plot point estimates (e.g. means) for each of several groups together with error bars representing confidence intervals.
- Use `plt.errorbar()` for this.

```
# compute mean, se, and CI half-width
df_bar = df.mean().reset_index()
df_bar.rename(columns={'index': 'variable', 0: 'mean'}, inplace=True)
df_se = (df.std() / np.sqrt(df.size)).reset_index()
df_se = df_se.rename(columns={'index': 'variable', 0: 'se'})

## figure 3
fig3, ax3 = plt.subplots(nrows=1, ncols=1)
_ = plt.errorbar(
    x=df_bar['mean'],
    y=df_bar['variable'],
    xerr=df_se['se'] * 1.96,
    fmt='o'
)
_ = ax3.set_xlabel('Mean and 95% CI (xx units)')
```

Error Bars

- You can use `plt.errorbar()` for both point and interval estimates as on the last slide, or set `fmt=None` to draw error bars only.
- Pass a tuple to `xerr` or `yerr` for asymmetric intervals.

```
## figure 4
fig4, ax4 = plt.subplots(nrows=1, ncols=1)
_ = plt.scatter(
    data=df_bar,
    x='mean',
    y='variable',
    marker='s',
    color='black'
)
_ = plt.errorbar(
    x=df_bar['mean'],
    y=df_bar['variable'],
    fmt=None,
    xerr=(df_se['se'] * 1.96, df_se['se'] * 1.96),
    ecolor='gray',
    capsize=4
)
_ = ax4.set_xlabel('Mean and 95% CI (xx units)')
```

Aesthetics

- In visualization (borrowing from the *grammar of graphics*) aesthetics refer to visual elements that can be mapped to data elements (or a fixed value).

Aesthetics

- Important aesthetics to make use of:
 - **marker** or shape of plotting point,
 - **color** for points, lines, and more, distinguish fill and edge colors,
 - **alpha** transparency level (especially useful when plot elements overlap),
 - **linestyle** the type or style of line to draw: solid, dashed, dotted, etc.
 - **markersize** the size of points to draw
- See **Line2D** properties at `plt.plot()` or refer to a matplotlib cheat sheet.

Pandas Plotting API

- Pandas simplifies creation of many basic plots when the plot data is in a pandas DataFrame or Series.
- Both have a `.plot()` method, which itself has a family of methods for specific types of plots.
- You can also use the `kind` parameter, but I prefer the methods for better interactive help.

Referencing Columns & Axes

- Refer to columns in the calling DataFrame using strings.
- Most plotting methods in panda accept a parameter **ax** you can use to specify the axes to plot on.

```
## figure 5
fig5, ax5 = plt.subplots(nrows=3, ncols=1, sharex=True)
fig5.tight_layout()
_ = df['x'].plot.hist(ax=ax5[0])
_ = df.plot.scatter('x', 'y', ax=ax5[1], color='darkgreen')
_ = df.plot.scatter('x', 'z', ax=ax5[2], color='red')
for i, lab in enumerate(['count', 'y', 'z']):
    _ = ax5[i].set_ylabel(lab)
```

Setup and Axes Handles

- Figures and axes will be setup automatically or will reference the current axes.
- The method calls will return an axis handle.

```
ax0 = df.plot.hist(color=['darkred', 'darkgreen', 'blue'], alpha=0.5)
_ = ax0.set_xlabel('x')
type(ax0)
```

Keyword Pass Through

- Keyword arguments will be passed through to the `plt` plotting function.

```
ax2 = df.plot.scatter(x='x', y='y', color='darkgreen', alpha=0.5, label='y')
_ = df.plot.scatter(ax=ax2, x='x', y='z', color='red', alpha=0.5, label='z')
_ = ax2.set_ylabel('y/z (DVs)')
_ = ax2.legend(loc='upper left')
```

Pandas or Pyplot?

- Prefer members of the pandas `.plot()` API when plot data is already in a DataFrame.
- Use pyplot to setup subplots and axes methods to customize.
- Your choice when (e.g. error bars) the interfaces don't align.

```
## figure 6 (like figure 4)
if not 'se' in df_bar:
    df_bar = pd.merge(df_bar, df_se, on='variable')
df_bar['moe'] = 1.96 * df_bar['se']

ax6 = df_bar.plot.scatter(
    x='mean',
    y='variable',
    marker='s',
    color='black'
)

_ = plt.errorbar(
    data=df_bar,
    x='mean',
    y='variable',
    fmt='None',
    xerr='moe',
    ecolor='gray',
    capsize=4
)

_ = ax6.set_xlabel('Mean and 95% CI (xx units)')
```

Plotting for Communication

- When adding plots to your notebooks or other summary documents, make your visualization professional and polished:
 - ensure axes have meaningful names and clear units,
 - think carefully about scales and organization,
 - use aesthetics such as color, shape, line type, size or facets thoughtfully and consistently,
 - use natural language and avoid programmer speak in labels, ticks, and legends.

Takeaways

- Visualization is a key part of data science.
- Matplotlib is the core visualization library in Python.
- Use parameter names for plot aesthetics over format string abbreviations.
- Pandas DataFrame and Series objects have useful plotting methods--prefer these.
- Polish your visualizations for better communication.